

第9组 StreamHub —— 前5天分工与任务 (8/25-8/29) 【5人小组】

前5天总分 40 分，8/29（周六）中期检查。核心目标：把上学期系统跑起来 + 补文档 + 补测试 + 容器化 + 跑通 CI/CD + 定微服务拆分方案。

一共 5 个角色，按「技术难度 + 工作量」尽量均衡分配。看完各自认领（可报第一/第二志愿），认领后在群里接龙回复：「姓名 + 角色编号」。

开工前现状（重要，影响工作量判断）

- ☒ Dockerfile（前端/后端）、docker-compose、数据库迁移脚本**已经存在**，不用从零写，但**尚未验证能否启动**。
- ☒ assets/ 已有上学期 3 份 PDF（需求/概要/开发计划）+ 5 份设计 MD，文档岗是**补**而不是从零。
- ☒ Git 已接好，`monolith-start` 标签已打在 `main` 上。后续所有人提交/推送统一推到 `main` 分支（标签是拆分前的固定起点快照，不随新提交移动），CI 由 push 到 `main` 触发。
- ☒ **目前仓库里没有真正的测试**（`test.js`/`test_simple.js` 是未提交的临时脚本），单元/集成/E2E 都要**从零写**。
- ☒ **没有任何 CI 配置**，流水线要**从零搭**。
- ☒ **仓库里有硬编码密钥/token，违反任务书「仓库不能出现密码/Token」**，第 1 天必须清：
 - `backend/docker-init/01_streamhub_backup.sql` 第 5 行的 `\restrict <64位 token>`（非法 `psql` 指令，会导致 `postgres` 初始化报错）；
 - `src/supabase/client.ts` 里的硬编码 `anon key`（Supabase 是模板残留死代码，可整体删掉）；
 - `backend/app/security.py` 的默认 `SECRET_KEY`，改由 `.env` 注入。
- ☒ **前端 7 处硬编码** `http://localhost:8000`，且 `webpack` 没有注入 `process.env`，部署/CI 时改不动，需要统一成 `API_BASE`。

五个角色（均衡版）

编号	角色	前5天要做什么	技术难度	工作量	适合谁
P1	组长 + 容器化	① 每天主持站会、写 200 字简报、维护看板、截图存档；② 列业务场景（用例）清单交老师确认（Git 已接、monolith-start 标签已打，无需重复）；③ 中期检查输出 微服务拆分方案 （服务划分图、接口清单、数据表归属表）；④ 容器化：验证/修补 Dockerfile、compose、DB 脚本，清除 \restrict token，保证三容器可启动、DB 自动初始化、README 可复现	中-高 ★★★★	中-高 ★★★	愿意牵头、会 Docker、看得懂后端架构
P2	文档A + E2E + 测试报告	① 写《软件需求规格说明书》《软件概要设计说明书》，画用例图、用例说明、概念类图、组件图、系统级+组件级顺序图；② 输出统一版本的 用例编号全集 ，交付 P3/P4/P5 使用；③ 用 Playwright 写 核心业务 E2E 测试 （覆盖全部用例主流程）；④ 汇总完整 测试报告 （合并 P4 的单元/集成结果 + 自己的 E2E 结果：总数/通过/失败/原因/环境）	中 ★★★	高 ★★★★	细心、会写文档画 UML、会一点 TS
P3	文档B + 追溯表	① 写《软件详细设计说明书》，画类图、对象级顺序图；② 维护 完整追溯表 （需求-用例-三层模型-代码-测试一张表），对接 P2 文档、P4/P2 测试，保证编号一一对应	中-高 ★★★★	高 ★★★★	熟悉项目代码，能画类图、对象级顺序图
P4	测试：单元 + API 集成	用 pytest 写后端单元测试、API 集成测试（FastAPI TestClient）；补断言、补异常分支，覆盖主/备/异常流程；把测试结果交给 P2 汇总进测试报告	中-高 ★★★★	中-高 ★★★	会 Python、pytest
P5	CI/CD + 公共前置清理	① 从零搭 GitHub Actions 流水线 （取码→装依赖→编译→单测→集成→制作镜像→部署 K8s→健康检查，测试失败必须停）；② 提交流水线配置、K8s 部署文件、测试/部署脚本；镜像用固定版本号，禁止 latest；③ 参与第 1 天公共风险清理：清理硬编码密钥、删除 supabase 残留代码，配合 P1 改前端硬编码 localhost 地址，统一为 API_BASE	高 ★★★★★	高 ★★★★	熟悉 GitHub Actions、Docker、K8s

说明：任务书 40 分 = 文档 10 + 测试 10 + 容器化 8 + CI/CD 10 + 中期 2。5 个人要把两个 10 分块各拆一半才能均衡：

- **文档 10** 拆成「需求+概要」(P2) 和「详细设计+追溯表」(P3)；

- **测试 10** 拆成「单元+API 集成」(P4) 和「E2E+测试报告」(P2, 与需求/概要合并在同一人, 因为 E2E 本质是翻译用例流程)。

对应 P1=10、P2=10、P3=5、P4=5、P5=10。P3/P4 分数偏轻, 作为机动兵力: P3 的追溯表本就是对文档和测试的枢纽, P4 的单元/集成结果喂给 P2 出报告, 两人再顺带支援 P2 画图、P5 冒烟测试。P1 的 10 分里「容器化」以验证修补为主 (Dockerfile/compose 已存在), 实际体力比 P2/P5 轻, 但背了微服务拆分方案 (中期硬要求) + 项目管理统筹。P5 的 CI/CD 是纯高难度单点, 卡住会拖累整条流水线。

5 天时间线 (谁哪天做什么)

日期	关键动作	主要责任
8/25 (第1天)	清 token/密钥 → 启动原系统 → 逐项检查用例 → 列用例清单交老师确认 → 建看板拉老师/助教	全组启动; P1 主导清单、看板 (Git/monolith-start 标签已就绪); P5 配合清密钥、硬编码地址
8/26-27 (第2-3天)	并行推进: P2 写需求概要 + 起 E2E; P3 做详细设计 + 追溯表; P4 写单元/API 集成测试; P5 搭 CI 流水线骨架; P1 验证容器 + 出微服务方案	各自开工, 互相对齐用例编号
8/28 (第4天)	CI 流水线完整跑通 (push 自动构建测试); 微服务方案定稿; P2 汇总测试报告; P3 校验追溯表	P5 调通流水线; P1 完善微服务方案; P2 出测试报告; P3 校验追溯表
8/29 (第5天)	中期检查: 核对 ① 系统能启动 + 用例可跑 + 有标签 ② 三层图 + 追溯表齐 ③ 容器能启动 + CI 能构建测试 ④ 微服务方案齐	全组逐项核对

协作约定 (避免各干各的)

1. **统一用例编号:** P2 输出一套标准用例编号, P2/P3 的图 (SYS-SEQ/COMP-SEQ/OBJ-SEQ)、P4 的单元/集成测试 (UNIT-TC/INT-TC)、P2 的 E2E (E2E-TC)、P3 的追溯表全部复用同一套编号, 否则中期检查「一一对应」过不了。
2. **测试命令契约:** P5 先定死 `pytest` (后端) 和 `npm test` (前端) 两个命令, P4 (单元/集成)、P2 (E2E) 照这套命令写, 互不阻塞。
3. **测试失败必须停:** 流水线任何一步失败 (尤其测试) 都不能继续构建镜像/部署, P2/P4/P5 提前约定退出码和失败信号。
4. **看板每天更新:** P1 维护看板, 但每人自己的任务卡要自己填「实际完成时间 + 证据链接」, 每天截看板图存档。
5. **第 1 天先清雷:** `\restrict` token、Supabase 死代码、硬编码密钥、前端硬编码 `localhost:8000` 是 P1/P5 的公共前置, 不清掉后面容器化和 CI 都会被卡。
6. **提交只进 main:** 所有代码统一提交并 push 到 `main` 分支, 不新建 feature 分支、不推 `master`; `monolith-start` 是已打好的固定标签, 任何人不得删除、移动或重新打。